

Whitepaper:

Smart Contract Design for MCP Utility (Market Cap Predictions)

1. Abstract

The MCP Utility smart contract, deployed on the Solana blockchain using the Anchor framework, enables a prediction market for \$AETHER's market capitalization. Users stake \$AETHER tokens to predict whether the market cap will exceed a target value by a specified deadline, earning MCP points for accurate predictions. These points can be redeemed for rewards, such as \$AETHER tokens or governance rights. The contract incorporates a burn mechanism for incorrect predictions, enhancing token economics, and ensures secure, transparent operations through access controls and Solana's SPL Token Program.

2. Introduction

2.1 Purpose

The MCP Utility facilitates a decentralized prediction market where participants stake \$AETHER to forecast the market capitalization of the \$AETHER token. The system incentivizes accurate predictions with MCP points, which can be redeemed for rewards, fostering engagement and aligning user incentives with token growth.

2.2 Objectives

- Enable users to stake \$AETHER on binary ("yes" or "no") market cap predictions.
- Distribute MCP points based on prediction accuracy.
- Implement a burn mechanism to reduce \$AETHER supply from incorrect predictions.
- Provide a secure, scalable, and transparent platform on Solana.
- Support reward redemption for \$AETHER tokens or governance rights.

2.3 Environment

- Blockchain: Solana, chosen for high throughput and low transaction costs.
- Framework: Anchor, a Rust-based framework for secure Solana program development.
- Token Standard: SPL Token Program for \$AETHER token management.

3. System Design

3.1 Key Features

- **Prediction Rounds:** Rounds are created with a target market cap and deadline. Users predict if \$AETHER's market cap will exceed the target by the deadline.
- **Staking Mechanism:** Users stake \$AETHER on "yes" or "no" predictions, with stakes locked in a program-controlled vault.
- **Round Finalization:** An admin or oracle updates the actual market cap to finalize rounds, triggering MCP distribution and burns.
- **MCP Points:** Awarded to users based on stake size, prediction accuracy, and a difficulty multiplier tied to the prediction timeline.
- **Reward Redemption:** MCP points can be redeemed for \$AETHER tokens (e.g., 1 MCP = 0.05 \$AETHER) or other rewards like governance rights.
- **Burn Mechanism:** A configurable percentage (e.g., 10%) of losing stakes is burned to reduce \$AETHER supply.

3.2 Accounts and Data Structures

The contract uses the following accounts to manage state and interactions:

3.2.1 Prediction Round Account

Stores details of each prediction round.

- **Fields:**
 - **round_id:** Unique identifier (u64).
 - **target_market_cap:** Target market cap in USD (u64, e.g., 50_000_000 for \$50M).
 - **deadline:** UNIX timestamp for round end (i64).
 - **actual_market_cap:** Actual market cap at deadline, set by admin/oracle (u64).
 - **is_finalized:** Finalization status (bool).
 - **total_yes_stake:** Total \$AETHER staked on "yes" (u64).
 - **total_no_stake:** Total \$AETHER staked on "no" (u64).

3.2.2 User Prediction Account

Tracks a user's stake and prediction for a round.

- Fields:
 - user: User's public key (Pubkey).
 - round_id: Associated round ID (u64).
 - prediction: "Yes" or "No" (enum: Yes, No).
 - stake_amount: \$AETHER staked (u64).
 - has_claimed: Whether MCP has been claimed (bool).

3.2.3 User MCP Account

Manages a user's MCP balance and redemption history.

- Fields:
 - user: User's public key (Pubkey).
 - mcp_balance: Current MCP balance (u64).
 - total_redeemed: Total MCP redeemed (u64).

3.2.4 Program State Account

Stores global program configuration.

- Fields:
 - admin: Admin's public key (Pubkey).
 - aether_vault: \$AETHER token vault public key (Pubkey).
 - reward_pool_balance: \$AETHER in the reward pool (u64).
 - burn_percentage: Percentage of losing stakes to burn (u8, e.g., 10).
 - mcp_per_stake: MCP reward per \$AETHER staked (u64, e.g., 0.5 MCP).
 - round_counter: Counter for round IDs (u64).

3.3 Instructions (Functions)

The smart contract exposes the following instructions:

3.3.1 Initialize Program

Sets up the program state and \$AETHER vault.

- Parameters:
 - admin: Admin's public key.
 - burn_percentage: Percentage of incorrect stakes to burn.
 - mcp_per_stake: MCP reward rate.
- Actions:
 - Initializes Program State Account.
 - Creates a token vault owned by the program.

3.3.2 Create Prediction Round

Creates a new prediction round.

- Parameters:
 - target_market_cap: Target market cap (USD).
 - deadline: UNIX timestamp for round end.
- Actions:
 - Initializes a Prediction Round Account.
 - Increments round_counter in Program State Account.

3.3.3 Stake Prediction

Allows users to stake \$AETHER on a prediction.

- Parameters:
 - round_id: Round ID.
 - prediction: "Yes" or "No".
 - amount: \$AETHER to stake.
- Actions:
 - Verifies round is active and not expired.
 - Transfers \$AETHER to the vault using SPL Token Program.

- Creates/updates User Prediction Account.
- Updates total_yes_stake or total_no_stake.

3.3.4 Finalize Round

Finalizes a round with the actual market cap.

- Parameters:
 - round_id: Round ID.
 - actual_market_cap: Actual market cap.
- Actions:
 - Restricts access to admin/oracle.
 - Sets is_finalized to true and stores actual_market_cap.
 - Burns a percentage of losing stakes (e.g., 10% of total_no_stake if market cap exceeds target).

3.3.5 Claim MCP

Distributes MCP points to users for a finalized round.

- Parameters:
 - round_id: Round ID.
- Actions:
 - Verifies round is finalized and user hasn't claimed.
 - Calculates MCP: $\text{stake_amount} * \text{mcp_per_stake} * \text{difficulty_multiplier}$.
 - $\text{difficulty_multiplier} = 1 + (\text{days_to_deadline} / 30)$.
 - Updates User MCP Account.

3.3.6 Redeem MCP

Redeems MCP for rewards.

- Parameters:
 - amount: MCP to redeem.
 - reward_type: Reward type (e.g., "aether_tokens").

- Actions:
 - Verifies sufficient mcp_balance.
 - For “aether_tokens”, transfers \$AETHER (1 MCP = 0.05 \$AETHER).
 - Updates mcp_balance and total_redeemed.

4. Security Considerations

4.1 Access Control

- Finalize Round: Restricted to the admin or a trusted oracle to prevent manipulation of actual_market_cap.
- Program State: Only the admin can initialize or modify critical settings (e.g., burn_percentage).

4.2 Token Transfers

- Uses SPL Token Program for secure \$AETHER transfers to/from the vault.
- Validates token accounts to prevent unauthorized transfers.

4.3 Rate Limiting

- Checks ensure users cannot stake after a round’s deadline or on finalized rounds.
- Prevents double-claiming of MCP rewards.

4.4 Burn Mechanism

- Configurable burn_percentage (e.g., 10%) is transparent and capped to avoid excessive burns.
- Burns are executed via SPL Token Program’s burn instruction.

4.5 Reward Pool Management

- Tracks reward_pool_balance to prevent over-redemption.
- Ensures sufficient \$AETHER in the vault before reward transfers.

5. Implementation Details

5.1 Solana and Anchor

- Solana: Provides high-speed, low-cost transactions ideal for frequent user interactions.
- Anchor: Simplifies account management, serialization, and error handling in Rust.

5.2 Example Workflow

- Admin initializes the program, setting `burn_percentage = 10` and `mcp_per_stake = 0.5`.
- Admin creates a round with `target_market_cap = $50M` and `deadline = 30 days`.
- User stakes 100 \$AETHER on “Yes” for `round_id = 1`.
- At deadline, admin finalizes the round with `actual_market_cap = $60M`.
- Losing “No” stakes are burned (10% of `total_no_stake`).
- User claims MCP: $100 * 0.5 * (1 + 30/30) = 100$ MCP.
- User redeems 100 MCP for 5 \$AETHER ($100 / 20$).

5.3 Error Handling

The contract defines custom errors, including:

- `RoundFinalized`: Attempt to stake on a finalized round.
- `RoundExpired`: Attempt to stake after deadline.
- `InsufficientMCP`: Attempt to redeem more MCP than available.
- `Unauthorized`: Non-admin access to restricted functions.

6. Future Enhancements

- **Oracle Integration**: Replace admin with a decentralized oracle (e.g., Chainlink) for `actual_market_cap`.
- **Additional Rewards**: Expand `reward_type` to include governance voting or NFTs.
- **Dynamic MCP Rates**: Adjust `mcp_per_stake` based on round participation or market conditions.
- **Frontend Integration**: Develop a user-friendly interface for staking, claiming, and redeeming.

7. Conclusion

The MCP Utility smart contract provides a robust, secure, and engaging prediction market for \$AETHER's market capitalization. By leveraging Solana's performance and Anchor's development tools, it ensures scalability and reliability. The burn mechanism and MCP rewards align user incentives with token growth, while security measures protect against manipulation. Future enhancements, such as oracle integration and expanded rewards, will further strengthen the platform's utility and adoption.

8. References

- Solana Documentation: <https://docs.solana.com>
- Anchor Framework: <https://www.anchor-lang.com>
- SPL Token Program: <https://spl.solana.com/token>

Note: The provided smart contract code has been redacted to exclude sensitive details (e.g., `declare_id!`) that could be used to manipulate the contract.